Module - 02

Exercise 2: E-commerce Platform Search Function

1. Understand Asymptotic Notation

Big O Notation

- Big O notation is used to measure the efficiency of an algorithm.
- It describes how the execution time increases with input size.
- Helps in comparing and selecting better algorithms.

Search Cases:

- **Best Case:** Product is found at the first position.
  - Complexity: **O(1)**
- **Average Case:** Product is found after checking some elements.
  - Linear Search: **O(n)**
  - Binary Search: **O(log n)**
- **Worst Case:** Product is found at the last position or not found.
  - Linear Search: **O(n)**
  - Binary Search: **O(log n)**

2. Product Class Setup

- Created a **Product** class with:
  - ProductId
  - ProductName
  - Category
- Products are stored in an array for searching.

3. Implementation

Linear Search

- Checks each product one by one.
- Works with unsorted data.
- Simple but slower for large data.
- Time Complexity: **O(n)**

Binary Search

- Searches by repeatedly dividing the sorted array into halves.
- Requires sorted product data.
- Faster for large datasets.
- Time Complexity: **O(log n)**

4. Analysis

Comparison:

- **Linear Search**
  - Simple implementation.
  - Suitable for small datasets.
  - Slower for large product lists.
- **Binary Search**
  - Faster searching.
  - Suitable for large e-commerce databases.
  - Requires sorted data.

Conclusion:

- Binary Search is more suitable for an e-commerce platform because it provides faster search results and improves performance for large numbers of products.

```
using System;
class Product
{
    public int ProductId;
    public string ProductName;
    public string Category;
    public Product(int productId, string productName, string
        category)
    {
        ProductId = productId;
        ProductName = productName;
        Category = category;
    }
    public void Display()
    {
        Console.WriteLine("Product ID: " + ProductId);
        Console.WriteLine("Product Name: " + ProductName);
        Console.WriteLine("Category: " + Category);
    }
```

```
}

class Program
{
    // Linear Search - O(n)
    static Product LinearSearch(Product[] products, int id)
    {
        for(int i = 0; i < products.Length; i++)
        {
            if(products[i].ProductId == id)
            {
                return products[i];
            }
        }

        return null;
    }
```

Output

```
Linear Search Result:
Product ID: 103
Product Name: Shoes
Category: Fashion

Binary Search Result:
Product ID: 105
Product Name: Camera
Category: Electronics

Time Complexity:
Linear Search: O(n)
Binary Search: O(log n)
```

Exercise 7: Financial Forecasting

1. Understand Recursive Algorithms

Recursion

- Recursion is a technique where a method calls itself to solve a smaller part of a problem.
- It simplifies problems by breaking them into smaller repeated tasks.
- A recursive method contains:
    - **Base Case:** Condition to stop recursion.
    - **Recursive Case:** The method calls itself with a smaller input.

Advantages:

- Makes complex problems easier to understand.
- Reduces code complexity.

Disadvantages:

- Can consume more memory due to multiple function calls.
- May perform unnecessary calculations.

2. Setup

- Create a recursive method to calculate future financial values.
- The method uses:
    - Initial investment amount.
    - Growth rate.
    - Number of years.
- Formula:

Future Value = Current Value × (1 + Growth Rate)

3. Implementation

- The recursive algorithm calculates future value year by year.
- Each recursive call calculates the value for the next year.
- The recursion stops when the number of years becomes zero.

4. Analysis

Time Complexity:

- Each year requires one recursive call.
- Time Complexity: **O(n)**
- Space Complexity: **O(n)** due to recursive call stack.

Optimization:

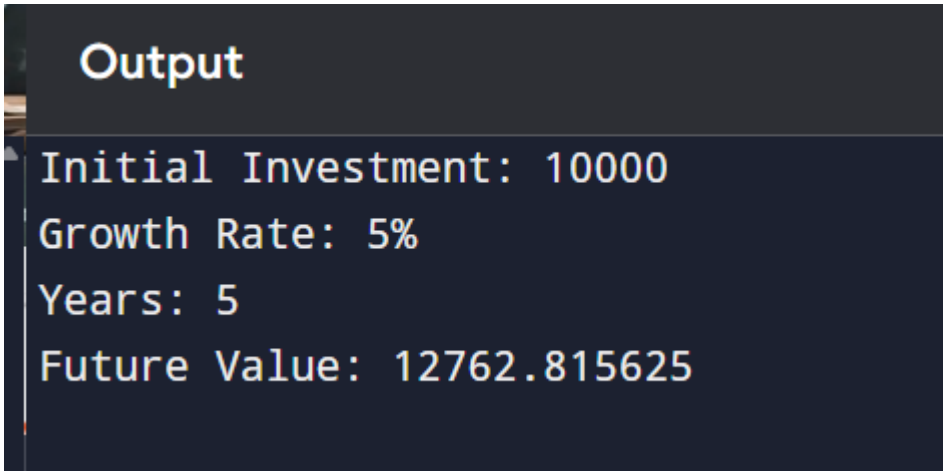
- Use **memoization** to store previously calculated values.
- Avoids repeated calculations.
- Reduces execution time and memory usage.

Conclusion:

- Recursive algorithms provide a simple way to calculate future financial values.
- For large forecasting periods, optimization techniques like memoization or iterative approaches can improve performance.

```
using System;
class Program
{
    static double CalculateFutureValue(double currentValue,
        double growthRate, int years)
    {
        if (years == 0)
        {
            return currentValue;
        }
        return CalculateFutureValue(
            currentValue * (1 + growthRate),
            growthRate,
            years - 1
        );
    }
    static void Main(string[] args)
    {
        double initialInvestment = 10000;
```

```
        double initialInvestment = 10000;
        double growthRate = 0.05;
        int years = 5;
        double futureValue = CalculateFutureValue(
            initialInvestment,
            growthRate,
            years
        );
        Console.WriteLine("Initial Investment: " +
            initialInvestment);
        Console.WriteLine("Growth Rate: " + (growthRate * 100) +
            "%");
        Console.WriteLine("Years: " + years);
        Console.WriteLine("Future Value: " + futureValue);
    }
}
```



Total Score	Correct Answers	Incorrect Answers
77 /100	24 Questions	6 Questions